



Fleet management web app

Submitted to: Dr Abdelrhman Abotaleb & Dr Ahmed Hussein

Done by: Mohamed Ahmed Abdelazim Mohamed

Deploy Fleet Management Web APP on AWS EC2

Launch an EC2 Instance

1. Go to EC2 Dashboard.
2. Click Launch Instance.
3. Choose an Amazon Machine Image (AMI):
 - Select Ubuntu 22.04 LTS.
4. Choose an Instance Type:
 - t2.micro(Free Tier).
5. Configure Security Group:
 - Allow Inbound Rules for:
 - SSH (22)
 - HTTP (80)
 - Custom TCP Rule (8501) → Required for Streamlit.

0.0.0.0/0 ✕

▼ Security group rule 4 (TCP, 8501, 0.0.0.0/0) Remove

Type Info	Protocol Info	Port range Info
Custom TCP	TCP	8501
Source type Info	Source Info	Description - optional Info
Custom	Add CIDR, prefix list or security group	e.g. SSH for admin desktop
	0.0.0.0/0 ✕	

⚠ Rules with source of 0.0.0.0/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only. ✕

Add security group rule

Figure 1 creating custom TCP rule for Streamlit

6. Click Launch Instance.

Connect to the machine and run these commands to install some dependencies:

```
sudo apt update && sudo apt upgrade -y  
sudo apt install python3-pip -y
```

Create virtual environment

```
sudo apt install python3-venv -y  
python3 -m venv venv  
source venv/bin/activate
```

install some required packages for the app

```
pip3 install streamlit pandas matplotlib plotly boto3 numpy joblib capstone
```

make this folder structure

```
└─ Fleet_Management  
  └─ app.py  
    └─ app_pages  
      └─ bootloader_overview.py  
      └─ fota.py  
      └─ introduction.py  
      └─ lsm_overview.py  
      └─ network_overview.py  
      └─ obd_overview.py  
      └─ someip_overview.py  
    └─ Bootloader  
      └─ cloud.py  
      └─ features.csv  
      └─ label_encoder.pkl  
      └─ malware_detector_model.pkl  
    └─ JSONS  
      └─ Firmware_v1.json  
      └─ Firmwasre_v2.json  
      └─ Firmware_v3.json  
  └─ Flask_Server  
    └─ received.py  
    └─ JSONS  
      └─ File1.json  
      └─ File2.json  
      └─ File3.json
```

Introduction.py

Purpose: This contains introduction for the pages in the Web app [3]

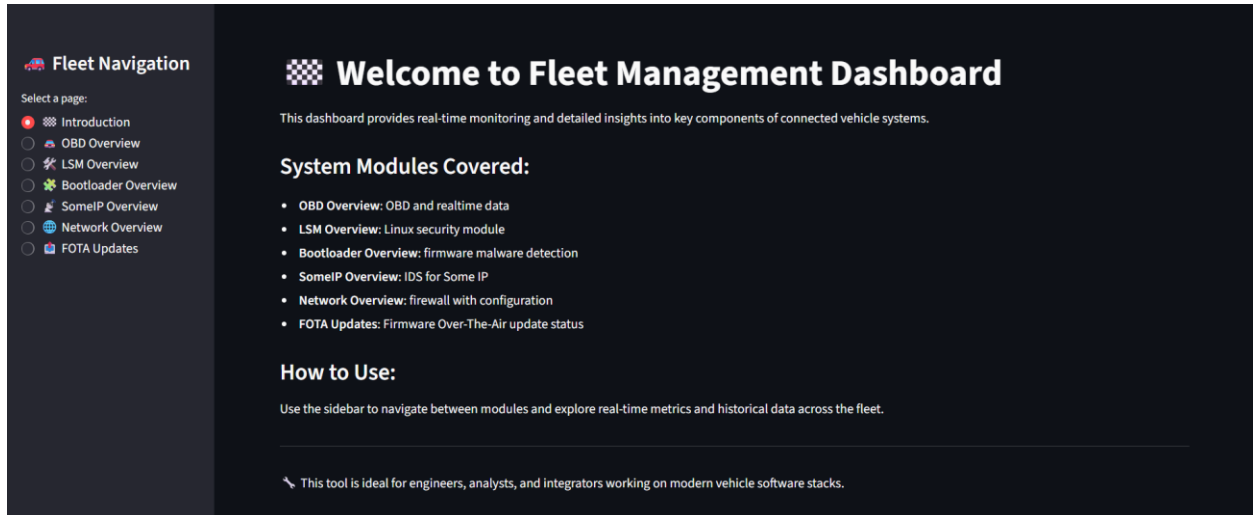


Figure 2 Introduction page

fota.py

Purpose: Allows users to upload firmware updates (in .bin format). [3]

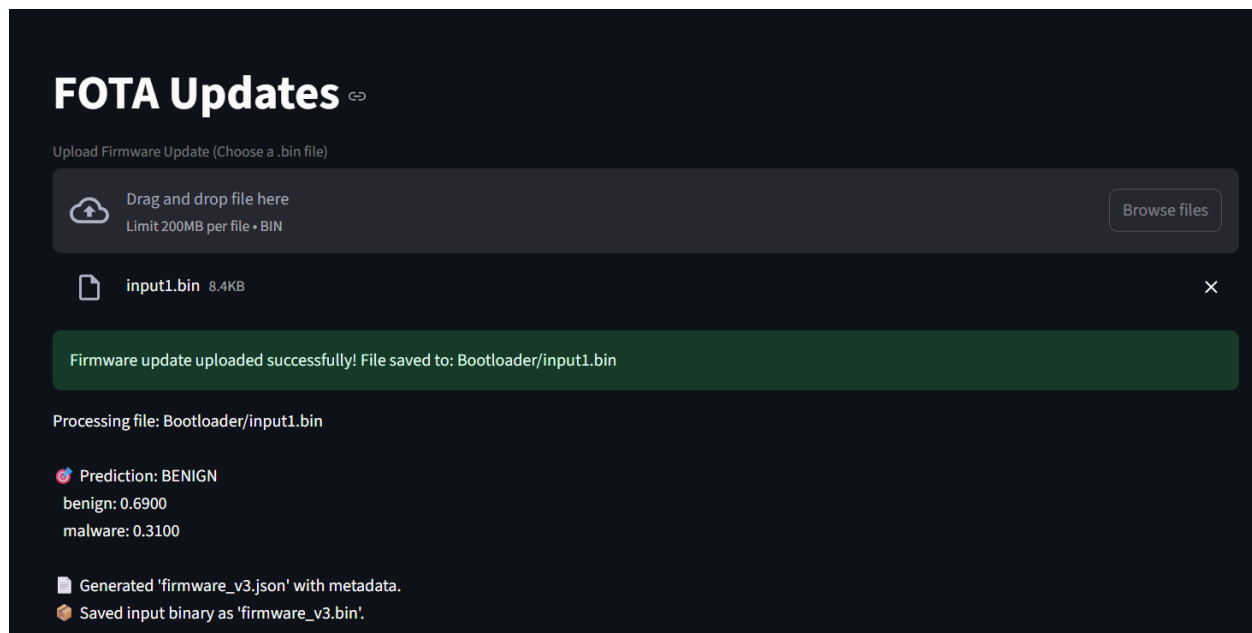


Figure 3 FOTA Updates page

Now to get the .bin file on our machine, run this command:

```
scp -i C:\Users\future\Desktop\WebApp\Web_app.pem  
ubuntu@13.58.250.233:~/uploads/debug_sample.bin .
```

This command will securely transfer the file debug_sample.bin from the remote server located at 3.139.97.26 (cloud server). The transfer will use the private key specified (Web_app.pem) for secure SSH authentication.

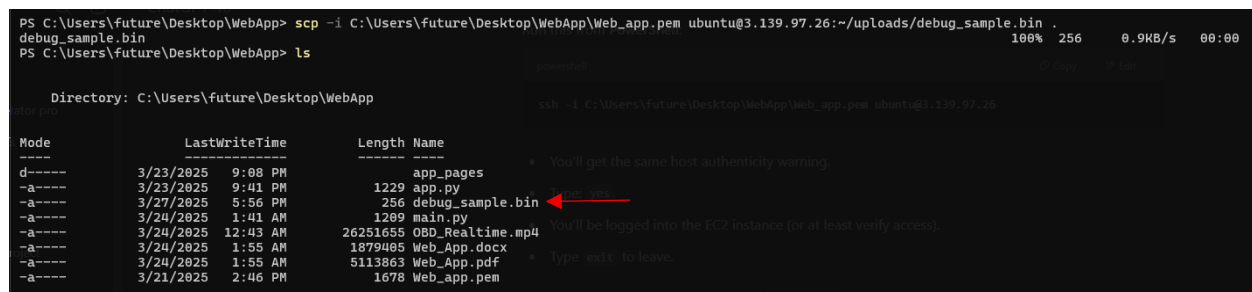


Figure 4 downloaded bin file

app.py

Purpose: This is the main app that serves as the central point for navigation within the Fleet Management Web App.

Used Libraries: Streamlit for the web interface, includes the various page modules like obd_overview, lsm_overview, bootloader_overview, etc. [3]

Run app.py

```
streamlit run app.py --server.port 8501
```

Open the app using this URL

```
http://<EC2_public_ip>:8501/
```

```
http://13.58.250.233:8501/
```

Now we need the app to Keep Running on the cloud machine, so we will use nohup command:

```
nohup streamlit run app.py --server.port 8501 &  
nohup python3 receiver.py &
```

We can stop the Streamlit app by killing the process using the port it's running on (8501):

- 1- The lsof (List Open Files) command helps identify the PID of the app running on a specific port (8501).

```
lsof -i :8501
```

- 2- Then kill it with its PID resulted from the previous command

```
kill <PID>
```

Sending Real-Time data to the Web App

1. Create a Timestream Database

Go to AWS Console → **Timestream**.

Click "**Create database**".

Name it: FleetManagementDB

Choose: **Standard database** → Click **Create**

2. Create a Timestream Table

In FleetManagementDB, click "Create table"

Name it: OBDDTable

Leave the default options as it is

Click Create table

3. Create a Thing (OBD Device)

Thing name: demo_thing

Attach this Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "iot:Publish",
        "iot:Subscribe",
        "iot:Connect",
        "iot:Receive"
      ],
      "Resource": "*"
    }
  ]
}
```

4. Create IoT Rule to Store data in Timestream

Name the Role: OBDThingtoTimeStream

Enter this SQL statement to get all the messages from the OBD topic.

```
SELECT * FROM 'obd_topic'
```


Action 1

▼

Timestream table

Write a message into a Timestream table

Database name

FleetManagementDB

▼

↺

↻

Create Timestream database

Table name

OBDDTable

▼

↺

↻

Create Timestream table

Dimensions

Each record contains an array of dimensions (minimum 1). Dimensions represent the metadata attributes of a time series data point.

Dimensions name

Dimension value

Car_ID

Car_ID

Remove

Location

Location

Remove

Board

Board

Remove

Error_Code

Error_Code

Remove

Add new dimension

Timestamp value - optional

Timestamp unit

#{time}

MILLISECONDS

IAM role

Choose a role to grant AWS IoT access to your endpoint.

demo_rule

▼

↺

↻

View

Create new role

Figure 5 Role parameters

5. Attach IAM Role to EC2 (Timestream Read Access)

Create an IAM Role (EC2TimestreamReader)

Attach this permission:

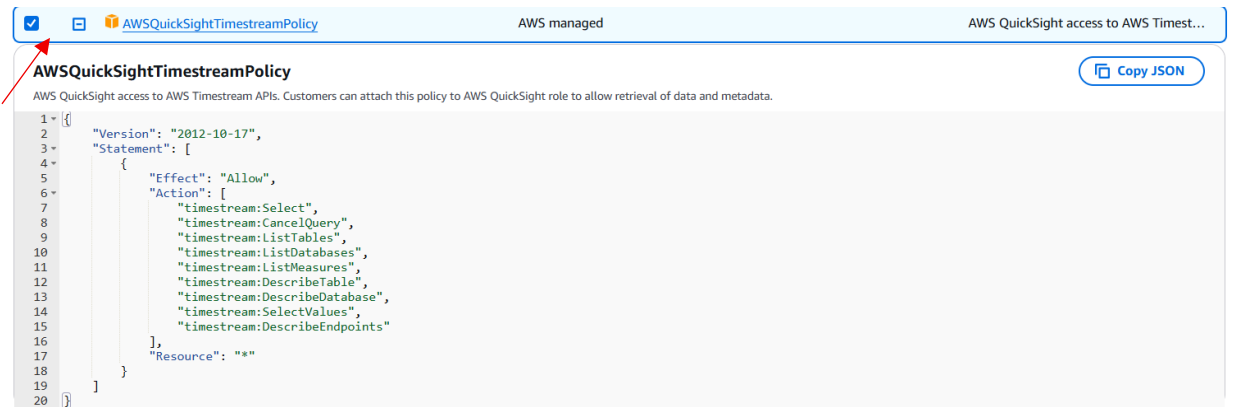


Figure 6 Policy for EC2 instance

Then to Attach this role to the EC2 instance:

- 1- under **Instance Settings**
- 2- Click Actions → Security → Modify IAM Role.
- 3- Select the IAM role that we have just created (EC2TimestreamReader).
- 4- Click Update IAM role.

Test the Fleet Management App

- 1- Run the C++ app that publish messages to AWS IoTCore Service
- 2- Apply MQTT test and make sure that messages are published correctly

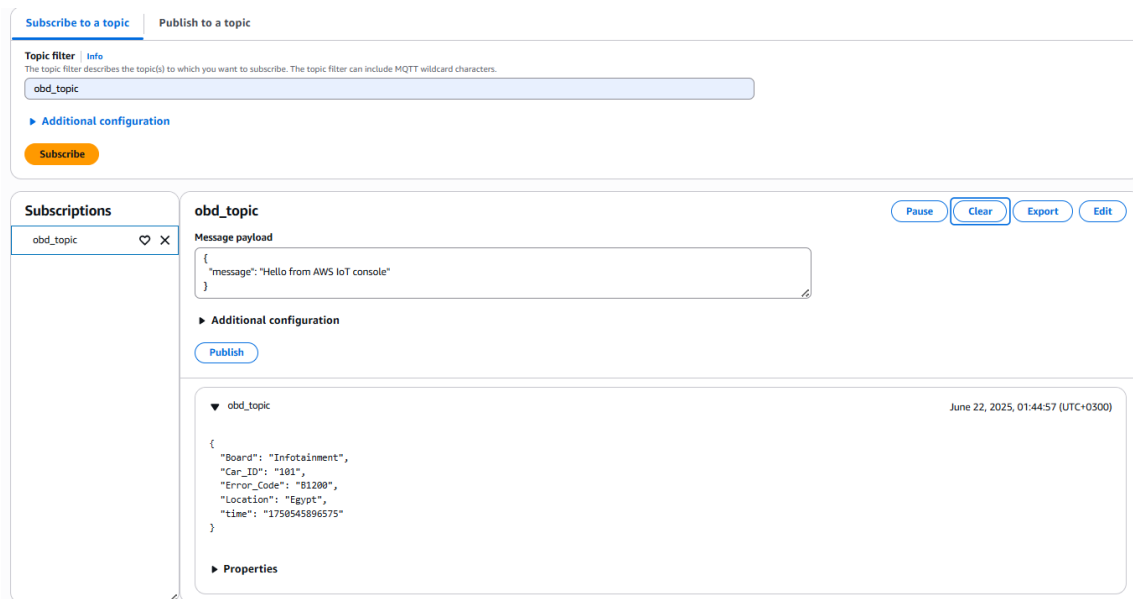


Figure 7 MQTT test client applied on OBD topic

Apply this Query test to make sure that Created role (transfer messages from IOTCore to Amazon Timestream DB) works properly:

```
SELECT
  time,
  MAX(CASE WHEN measure_name = 'Car_ID' THEN measure_value::varchar END) AS Car_ID,
  MAX(CASE WHEN measure_name = 'Location' THEN measure_value::varchar END) AS
Location,
  MAX(CASE WHEN measure_name = 'Board' THEN measure_value::varchar END) AS Board,
  MAX(CASE WHEN measure_name = 'Error_Code' THEN measure_value::varchar END) AS
Error_Code
FROM "FleetManagementDB"."OBDDTable"
GROUP BY time
ORDER BY time DESC
```

```

1 SELECT
2   time,
3   MAX(CASE WHEN measure_name = 'Car_ID' THEN measure_value::varchar END) AS Car_ID,
4   MAX(CASE WHEN measure_name = 'Location' THEN measure_value::varchar END) AS Location,
5   MAX(CASE WHEN measure_name = 'Board' THEN measure_value::varchar END) AS Board,
6   MAX(CASE WHEN measure_name = 'Error_Code' THEN measure_value::varchar END) AS Error_Code
7 FROM "FleetManagementDB"."OBDDTable"
8 GROUP BY time
9 ORDER BY time DESC
10

```

Save Clear Run ☐ Enable Insights

Table details **Query results** Output

Rows returned (4)

time	Car_ID	Location	Board	Error_Code
2025-06-21 22:44:56.575000000	101	Egypt	Infotainment	B1200
2025-06-21 16:50:11.804000000	101	Egypt	Infotainment	B1200
2025-06-21 16:38:56.435000000	101	Egypt	Infotainment	B1200
2025-06-21 15:41:12.546000000	101	Egypt	Infotainment	B1200

Figure 8 Query Results

Then we have created a script that creates a dashboard that displays live OBD error data retrieved from AWS Timestream database. It fetches the latest OBD logs, including car ID, location, error code, and timestamp, board type, and displays them in a tabular format with expandable entries for each log. [3]

Used Libraries:

Streamlit: For creating the web app interface and displaying the data.

boto3: For interacting with AWS Timestream, querying the database, and fetching the data.

pandas: for data handling.

```

import streamlit as st
import boto3
import pandas as pd
from botocore.exceptions import BotoCoreError, ClientError

st.title("🚗 Live OBD Error Dashboard")

# Initialize Timestream client
try:

```

```

client = boto3.client('timestream-query', region_name='us-east-2')
except (BotoCoreError, ClientError) as e:
    st.error(f'Failed to create Timestream client: {e}')
    st.stop()

# SQL Query
QUERY = """
SELECT
    time,
    MAX(CASE WHEN measure_name = 'Car_ID' THEN measure_value::varchar END) AS Car_ID,
    MAX(CASE WHEN measure_name = 'Location' THEN measure_value::varchar END) AS
Location,
    MAX(CASE WHEN measure_name = 'Error_Code' THEN measure_value::varchar END) AS
Error_Code
FROM "FleetManagementDB"."FleetManagementDBTable"
GROUP BY time
ORDER BY time DESC
LIMIT 50
"""

# Execute query
try:
    response = client.query(QueryString=QUERY)
    rows = response.get('Rows', [])
    columns = [col['Name'] for col in response.get('ColumnInfo', [])]

    if not rows or not columns:
        st.warning("No data returned from Timestream.")
        st.stop()

    # Parse rows
    data = [[col.get('ScalarValue', '') for col in row['Data']] for row in rows]
    df = pd.DataFrame(data, columns=columns)

except (BotoCoreError, ClientError) as e:
    st.error(f'Query failed: {e}')
    st.stop()

# Display table
st.subheader("🔍 Latest OBD Logs")
st.dataframe(df)

# Expanders for details
for _, row in df.iterrows():
    with st.expander(f"🚗 {row['Car_ID']} - {row['Error_Code']}"):
        st.write(f"📍 **Location:** {row['Location']}")

```

```
st.write(f" **Time:** {row['time']}")
```

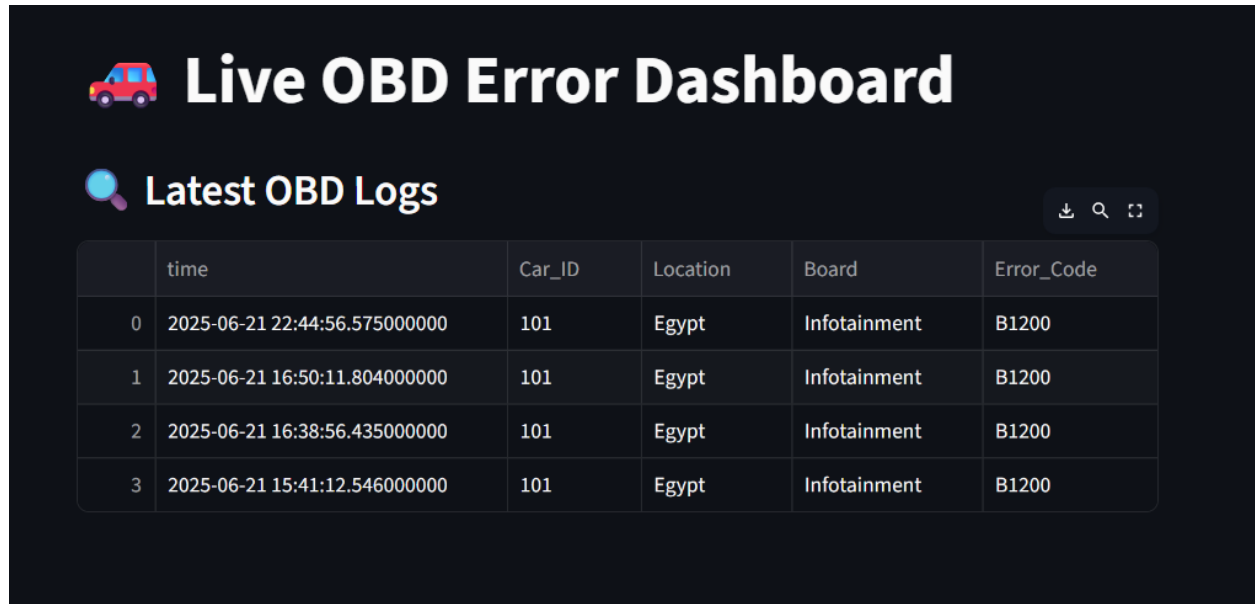


Figure 9 Final Results

The following link provides a demonstration video showcasing the application in operation:

<https://drive.google.com/file/d/1qDM56tukEz2kaKOdhmk4qCfGnKTPpnH/view?usp=sharing>

Creating multiple things for system components

Sending LSM Real-Time data to the Web App

1. Choose FleetManagementDB

2. Create a Timestream Table

In FleetManagementDB, click "Create table"

Name it: LSMTable

Leave the default options as it is

Click Create table

3. Create a Thing (LSM app)

Thing name: LSM_thing

Attach Iot Policy

4. Create IoT Rule to Store data in Timestream

Name the Role: LSMThingtoTimeStream

Enter this SQL statement to get all the messages from the LSM topic.

```
SELECT * FROM 'lsm_topic'
```

Action 1

▼

Timestream table

Write a message into a Timestream table

▼

Remove

Database name

Info

FleetManagementDB

▼

↺

↻

Create Timestream database

Table name

LSMTable

▼

↺

↻

Create Timestream table

Dimensions

Each record contains an array of dimensions (minimum 1). Dimensions represent the metadata attributes of a time series data point.

Dimensions name	Dimension value	
Car_ID	\${Car_ID}	Remove
Object	\${Object}	Remove
Board	\${Board}	Remove
Subject	\${Subject}	Remove
Access	\${Access}	Remove
Status	\${Status}	Remove

Add new dimension

Timestamp value - optional

\$(time)

▼

Timestamp unit

MILLISECONDS

▼

IAM role

Choose a role to grant AWS IoT access to your endpoint.

demo_rule

▼

↺

↻

View

Create new role

AWS IoT will automatically create a policy with a prefix of "aws-iot-rule" under your IAM role selected.

Figure 10 Role parameters

Test the Fleet Management App

- 3- Run the C++ app that publish messages to AWS IoTCore Service
- 4- Apply MQTT test and make sure that messages are published correctly

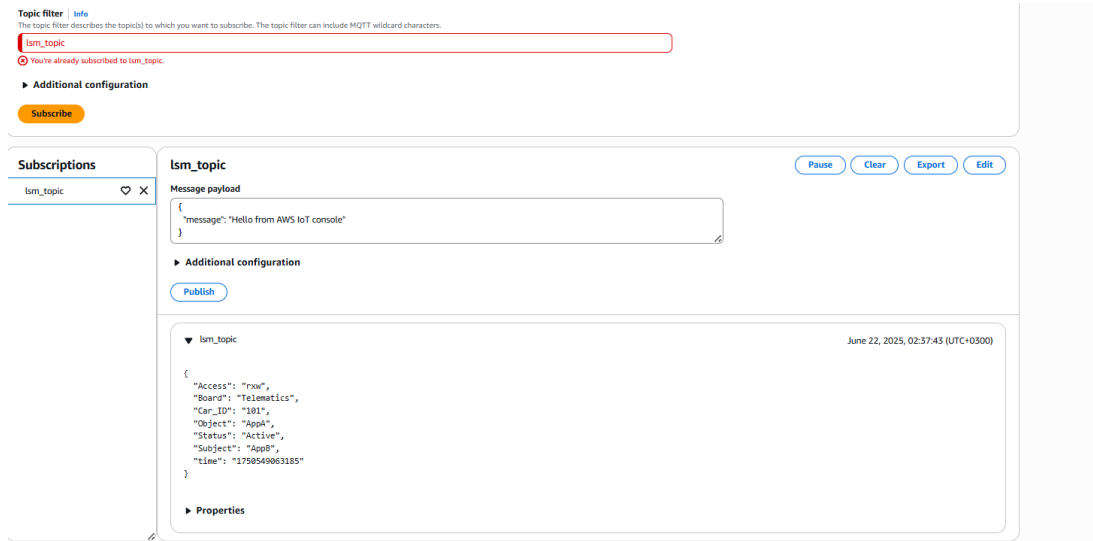


Figure 11 MQTT test client

Apply this Query test to make sure that Created role (transfer messages from IOTCore to Amazon Timestream DB) works properly:

```
SELECT
  time,
  MAX(CASE WHEN measure_name = 'Car_ID' THEN measure_value::varchar END) AS Car_ID,
  MAX(CASE WHEN measure_name = 'Object' THEN measure_value::varchar END) AS Object,
  MAX(CASE WHEN measure_name = 'Board' THEN measure_value::varchar END) AS Board,
  MAX(CASE WHEN measure_name = 'Subject' THEN measure_value::varchar END) AS Subject,
  MAX(CASE WHEN measure_name = 'Access' THEN measure_value::varchar END) AS Access,
  MAX(CASE WHEN measure_name = 'Status' THEN measure_value::varchar END) AS Status
FROM "FleetManagementDB"."LSMTable"
GROUP BY time
ORDER BY time DESC
GROUP BY time
ORDER BY time DESC
```

Query 1

```

1 SELECT
2   time,
3   MAX(CASE WHEN measure_name = 'Car_ID' THEN measure_value::varchar END) AS Car_ID,
4   MAX(CASE WHEN measure_name = 'Object' THEN measure_value::varchar END) AS Object,
5   MAX(CASE WHEN measure_name = 'Board' THEN measure_value::varchar END) AS Board,
6   MAX(CASE WHEN measure_name = 'Subject' THEN measure_value::varchar END) AS Subject,
7   MAX(CASE WHEN measure_name = 'Access' THEN measure_value::varchar END) AS Access,
8   MAX(CASE WHEN measure_name = 'Status' THEN measure_value::varchar END) AS Status
9 FROM "FleetManagementDB"."LSMTable"
10 GROUP BY time
11 ORDER BY time DESC
12

```

Save Clear Run Enable Insights

Table details Query results Output

Rows returned (1)

time	Car_ID	Object	Board	Subject	Access	Status
2025-06-21 23:51:39.219000000	101	AppA	Telematics	AppB	rxw	Active

Figure 12 Query Results

Then we have created a script that creates a dashboard that displays live LSM data retrieved from AWS Timestream database.

```

import streamlit as st
import boto3
import pandas as pd
from botocore.exceptions import BotoCoreError, ClientError

st.title("🚗 Live LSM Dashboard")

# Initialize Timestream client
try:
    client = boto3.client('timestream-query', region_name='us-east-2')
except (BotoCoreError, ClientError) as e:
    st.error(f"Failed to create Timestream client: {e}")
    st.stop()

# SQL Query
QUERY = """
SELECT
    time,
    MAX(CASE WHEN measure_name = 'Car_ID' THEN measure_value::varchar END) AS Car_ID,
    MAX(CASE WHEN measure_name = 'Object' THEN measure_value::varchar END) AS Object,
    MAX(CASE WHEN measure_name = 'Board' THEN measure_value::varchar END) AS Board,
    MAX(CASE WHEN measure_name = 'Subject' THEN measure_value::varchar END) AS Subject,
    MAX(CASE WHEN measure_name = 'Access' THEN measure_value::varchar END) AS Access,
    MAX(CASE WHEN measure_name = 'Status' THEN measure_value::varchar END) AS Status
FROM "FleetManagementDB"."LSMTable"

```

```

GROUP BY time
ORDER BY time DESC
"""

# Execute query
try:
    response = client.query(QueryString=QUERY)
    rows = response.get('Rows', [])
    columns = [col['Name'] for col in response.get('ColumnInfo', [])]

    if not rows or not columns:
        st.warning("No data returned from Timestream.")
        st.stop()

    # Parse rows
    data = [[col.get('ScalarValue', "") for col in row['Data']] for row in rows]
    df = pd.DataFrame(data, columns=columns)

except (BotoCoreError, ClientError) as e:
    st.error(f"Query failed: {e}")
    st.stop()

# Display table
st.subheader("🔍 Latest LSM Logs")
st.dataframe(df)

```

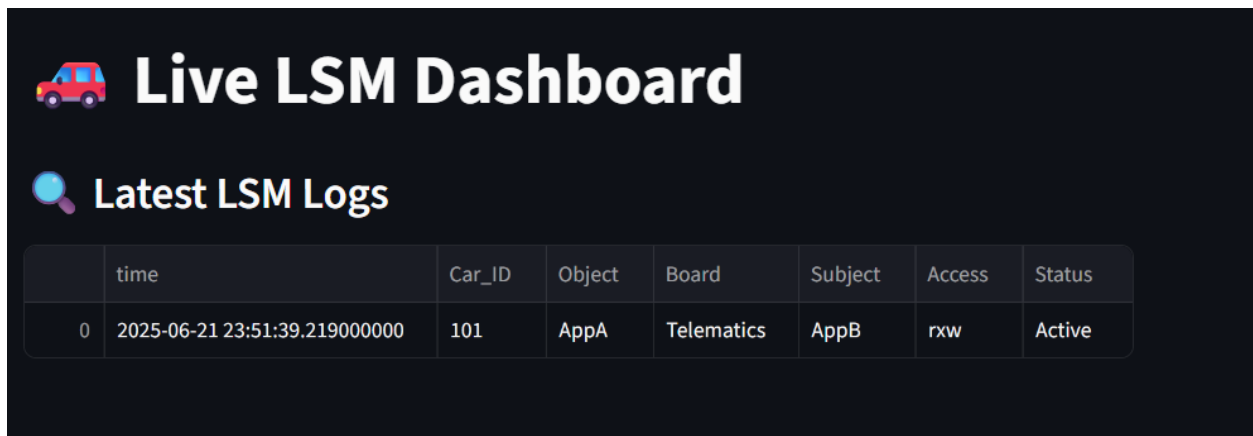


Figure 13 Final Results

Sending BootLoader Real-Time data to the Web App

1. Choose FleetManagementDB

2. Create a Timestream Table

In FleetManagementDB, click "Create table"

Name it: BLTable

Leave the default options as it is

Click Create table

3. Create a Thing (BL app)

Thing name: BL_thing

Attach Iot Policy

4. Create IoT Rule to Store data in Timestream

Name the Role: BLThingtoTimeStream

Enter this SQL statement to get all the messages from the BL topic.

```
SELECT * FROM 'BL_topic'
```

Action 1

▼

Timestream table

Write a message into a Timestream table

▼

Remove

Database name

Info

FleetManagementDB

▼

↺

↻

Create Timestream database

Table name

BLTable

▼

↺

↻

Create Timestream table

Dimensions

Each record contains an array of dimensions (minimum 1). Dimensions represent the metadata attributes of a time series data point.

Dimensions name

Dimension value

Car_ID

\$(Car_ID)

Remove

FileName

\$(FileName)

Remove

Board

\$(Board)

Remove

Approved

\$(Approved)

Remove

Label

\$(Label)

Remove

Add new dimension

Timestamp value - optional

Timestamp unit

\$(time)

▼

MILLISECONDS

▼

IAM role

Choose a role to grant AWS IoT access to your endpoint.

demo_rule

▼

↺

View

Create new role

AWS IoT will automatically create a policy with a prefix of "aws-iot-rule" under your IAM role selected.

Figure 14 Role parameters

20 | Page

Test the Fleet Management App

- 1- Run the C++ app that publish messages to AWS IoTCore Service
- 2- Apply MQTT test and make sure that messages are published correctly

The screenshot shows the AWS IoT Core MQTT test client interface. It is divided into two main sections: "Subscribe to a topic" and "Publish to a topic".

Subscribe to a topic:

- Topic filter:** BL_topic
- Additional configuration:** (Expanded)
- Subscribe:** (Button)

Publish to a topic:

- Message payload:** { "message": "Hello from AWS IoT console" }
- Additional configuration:** (Expanded)
- Publish:** (Button)
- Subscriptions:** (List on the left showing BL_topic)
- Message history:** (Expanded for BL_topic, showing a message received on June 22, 2025, 03:26:01 (UTC+0300) with payload: { "Approved": "1", "Board": "Telematics", "Car_ID": "101", "Filename": "Firm_v2.bin", "Label": "safe", "time": "1758551961053" })
- Properties:** (Expanded)

Figure 15 MQTT test client

Apply this Query test to make sure that Created role (transfer messages from IOTCore to Amazon Timestream DB) works properly:

```
SELECT
  time,
  MAX(CASE WHEN measure_name = 'Car_ID' THEN measure_value::varchar END) AS Car_ID,
  MAX(CASE WHEN measure_name = 'FileName' THEN measure_value::varchar END) AS
FileName,
  MAX(CASE WHEN measure_name = 'Board' THEN measure_value::varchar END) AS Board,
  MAX(CASE WHEN measure_name = 'Approved' THEN measure_value::varchar END) AS
Approved,
  MAX(CASE WHEN measure_name = 'Label' THEN measure_value::varchar END) AS Label
FROM "FleetManagementDB"."BLTable"
GROUP BY time
ORDER BY time DESC
```

The screenshot shows a SQL query editor with a query that selects various fields from a table in the FleetManagementDB database. The query results are displayed in a table with 6 columns: time, Car_ID, FileName, Board, Approved, and Label. The results show a single row of data for the time 2025-06-22 00:26:01.0530 00000, with Car_ID 101, FileName firm_v2.bin, Board Telematics, Approved 1, and Label safe.

```

1 SELECT
2   time,
3   MAX(CASE WHEN measure_name = 'Car_ID' THEN measure_value::varchar END) AS Car_ID,
4   MAX(CASE WHEN measure_name = 'FileName' THEN measure_value::varchar END) AS FileName,
5   MAX(CASE WHEN measure_name = 'Board' THEN measure_value::varchar END) AS Board,
6   MAX(CASE WHEN measure_name = 'Approved' THEN measure_value::varchar END) AS Approved,
7   MAX(CASE WHEN measure_name = 'Label' THEN measure_value::varchar END) AS Label
8 FROM "FleetManagementDB"."BLTable"
9 GROUP BY time
10 ORDER BY time DESC
11

```

Save Clear Run Enable Insights

Table details Query results Output

Rows returned (1)

time	Car_ID	FileName	Board	Approved	Label
2025-06-22 00:26:01.0530 00000	101	firm_v2.bin	Telematics	1	safe

Figure 16 Query Results

Then we have created a script that creates a dashboard that displays live Bootloader data retrieved from AWS Timestream database.

```

import streamlit as st
import boto3
import pandas as pd
from botocore.exceptions import BotoCoreError, ClientError

st.title("🚗 Live BootLoader Dashboard")

# Initialize Timestream client
try:
    client = boto3.client('timestream-query', region_name='us-east-2')
except (BotoCoreError, ClientError) as e:
    st.error(f'Failed to create Timestream client: {e}')
    st.stop()

# SQL Query
QUERY = """
SELECT
    time,
    MAX(CASE WHEN measure_name = 'Car_ID' THEN measure_value::varchar END) AS Car_ID,
    MAX(CASE WHEN measure_name = 'FileName' THEN measure_value::varchar END) AS
FileName,
    MAX(CASE WHEN measure_name = 'Board' THEN measure_value::varchar END) AS Board,
    MAX(CASE WHEN measure_name = 'Approved' THEN measure_value::varchar END) AS
Approved,

```

```

MAX(CASE WHEN measure_name = 'Label' THEN measure_value::varchar END) AS Label
FROM "FleetManagementDB"."BLTable"
GROUP BY time
ORDER BY time DESC
"""

# Execute query
try:
    response = client.query(QueryString=QUERY)
    rows = response.get('Rows', [])
    columns = [col['Name'] for col in response.get('ColumnInfo', [])]

    if not rows or not columns:
        st.warning("No data returned from Timestream.")
        st.stop()

    # Parse rows
    data = [[col.get('ScalarValue', '') for col in row['Data']] for row in rows]
    df = pd.DataFrame(data, columns=columns)

except (BotoCoreError, ClientError) as e:
    st.error(f"Query failed: {e}")
    st.stop()

# Display table
st.subheader("🔍 Latest BootLoader Logs")
st.dataframe(df)

```

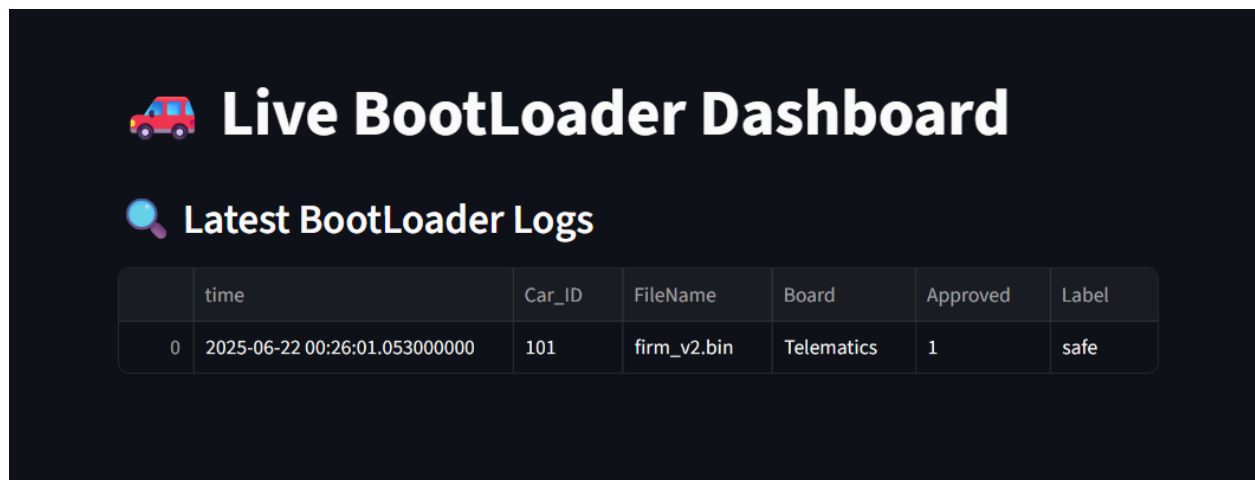


Figure 17 Final Results

Sending SomeIP 1 Real-Time data to the Web App

1. Choose FleetManagementDB

2. Create a Timestream Table

In FleetManagementDB, click "Create table"

Name it: SIPOneTable

Leave the default options as it is

Click Create table

3. Create a Thing (SIPOne app)

Thing name: SIPOne_thing

Attach Iot Policy

4. Create IoT Rule to Store data in Timestream

Name the Role: SIPOneThingtoTimeStream

Enter this SQL statement to get all the messages from the SomeIPone topic.

```
SELECT * FROM 'SIPOne_topic'
```

Action 1

▼ **Timestream table**
Write a message into a Timestream table ▼ [Remove](#)

Database name [Info](#)
FleetManagementDB [Refresh](#) [Help](#) [Create Timestream database](#)

Table name
SIPOneTable [Refresh](#) [Help](#) [Create Timestream table](#)

Dimensions
Each record contains an array of dimensions (minimum 1). Dimensions represent the metadata attributes of a time series data point.

Dimensions name	Dimension value	
Car_ID	\${Car_ID}	Remove
AttackSummary	\${AttackSummary}	Remove
Board	\${Board}	Remove

[Add new dimension](#)

Timestamp value - optional
\${time}

Timestamp unit
MILLISECONDS ▼

IAM role
Choose a role to grant AWS IoT access to your endpoint.
demo_rule [Refresh](#) [View](#) [Create new role](#)

AWS IoT will automatically create a policy with a prefix of "aws-iot-rule" under your IAM role selected.

Figure 18 Role parameters

Test the Fleet Management App

- 1- Run the C++ app that publish messages to AWS IOTCore Service
- 2- Apply MQTT test and make sure that messages are published correctly

Topic filter [Info](#)
The topic filter describes the topic(s) to which you want to subscribe. The topic filter can include MQTT wildcard characters.
SIPOne_topic

[Additional configuration](#)

[Subscribe](#)

Subscriptions

BL_topic	♥	×
SIPOne_topic	♥	×

SIPOne_topic [Pause](#) [Clear](#) [Export](#) [Edit](#)

Message payload

```
{
  "message": "Hello from AWS IoT console"
}
```

Additional configuration
[Publish](#)

▼ SIPOne_topic June 22, 2025, 03:47:40 (UTC+0300)

```
{
  "AttackSummary": "Suspicious Login Attempt_1",
  "Board": "Telematics",
  "Car_ID": "101",
  "time": "1750553259478"
}
```

Properties

Figure 19 MQTT test client

Apply this Query test to make sure that Created role (transfer messages from IOTCore to Amazon Timestream DB) works properly:

```
SELECT
  time,
  MAX(CASE WHEN measure_name = 'Car_ID' THEN measure_value::varchar END) AS Car_ID,
  MAX(CASE WHEN measure_name = 'AttackSummary' THEN measure_value::varchar END) AS
AttackSummary,
  MAX(CASE WHEN measure_name = 'Board' THEN measure_value::varchar END) AS Board
FROM "FleetManagementDB"."SIPOneTable"
GROUP BY time
ORDER BY time DESC
```

The screenshot shows a query execution interface. At the top, there's a tab labeled "Query 1". Below it, the SQL query is displayed in a code editor. The query is a SELECT statement with four columns: time, Car_ID, AttackSummary, and Board. It filters data from the "FleetManagementDB"."SIPOneTable" and groups the results by time, ordering them by time in descending order. Below the code editor, there are buttons for "Save", "Clear", and "Run", along with a checkbox for "Enable Insights". The "Query results" tab is selected, showing a table with one row of data. The table has four columns: time, Car_ID, AttackSummary, and Board. The data row shows a timestamp of "2025-06-22 00:49:28.3360 00000", a Car_ID of "101", an AttackSummary of "Suspicious Login Attempt_1", and a Board of "Telematics".

```
1 SELECT
2   time,
3   MAX(CASE WHEN measure_name = 'Car_ID' THEN measure_value::varchar END) AS Car_ID,
4   MAX(CASE WHEN measure_name = 'AttackSummary' THEN measure_value::varchar END) AS AttackSummary,
5   MAX(CASE WHEN measure_name = 'Board' THEN measure_value::varchar END) AS Board
6 FROM "FleetManagementDB"."SIPOneTable"
7 GROUP BY time
8 ORDER BY time DESC
9
```

Save Clear Run ☐ Enable Insights

Table details **Query results** Output

Rows returned (1)

time	Car_ID	AttackSummary	Board
2025-06-22 00:49:28.3360 00000	101	Suspicious Login Attempt_1	Telematics

Figure 20 Query Results

Sending SomeIP 2 Real-Time data to the Web App

1. Choose FleetManagementDB

2. Create a Timestream Table

In FleetManagementDB, click "Create table"

Name it: SIPTwoTable

Leave the default options as it is

Click Create table

3. Create a Thing (SIPTwo Device)

Thing name: SIPTwo_thing

Attach Iot Policy

4. Create IoT Rule to Store data in Timestream

Name the Role: SIPTwoThingtoTimeStream

Enter this SQL statement to get all the messages from the SomeIP 2 topic.

```
SELECT * FROM 'SIPTwo_topic'
```

Action 1

▼ Timestream table Remove
Write a message into a Timestream table

Database name [Info](#)
FleetManagementDB ↺ ↻ ↗ [Create Timestream database](#) ↗

Table name
SIPTwoTable ↺ ↻ ↗ [Create Timestream table](#) ↗

Dimensions
Each record contains an array of dimensions (minimum 1). Dimensions represent the metadata attributes of a time series data point.

Dimensions name	Dimension value	
Car_ID	\${Car_ID}	Remove
AttackSummary	\${AttackSummary}	Remove
Board	\${Board}	Remove

[Add new dimension](#)

Timestamp value - optional **Timestamp unit**
 \${time} MILLISECONDS ▼

IAM role
Choose a role to grant AWS IoT access to your endpoint.
 demo_rule ↺ ↻ ↗ [View](#) ↗ [Create new role](#)

AWS IoT will automatically create a policy with a prefix of "aws-iot-rule" under your IAM role selected.

Figure 21 Role parameters

Test the Fleet Management App

- 1- Run the C++ app that publish messages to AWS IOTCore Service
- 2- Apply MQTT test and make sure that messages are published correctly

Topic filter [Info](#)
The topic filter describes the topic(s) to which you want to subscribe. The topic filter can include MQTT wildcard characters.

SIPTwo_topic

► Additional configuration [Subscribe](#)

Subscriptions

SIPTwo_topic ♥ ✕

SIPTwo_topic [Pause](#) [Clear](#) [Export](#) [Edit](#)

Message payload

```
{
  "message": "Hello from AWS IoT console"
}
```

► Additional configuration [Publish](#)

▼ SIPTwo_topic June 22, 2025, 04:36:33 (UTC+0300)

```
{
  "AttackSummary": "Unauthorized Access Detected_2",
  "Board": "Telematics",
  "Car_ID": "101",
  "time": "1750556191770"
}
```

► Properties

Figure 22 MQTT test client

Apply this Query test to make sure that Created role (transfer messages from IOTCore to Amazon Timestream DB) works properly:

```
SELECT
  time,
  MAX(CASE WHEN measure_name = 'Car_ID' THEN measure_value::varchar END) AS Car_ID,
  MAX(CASE WHEN measure_name = 'AttackSummary' THEN measure_value::varchar END) AS
AttackSummary,
  MAX(CASE WHEN measure_name = 'Board' THEN measure_value::varchar END) AS Board
FROM "FleetManagementDB"."SIPOneTable"
GROUP BY time
ORDER BY time DESC
```

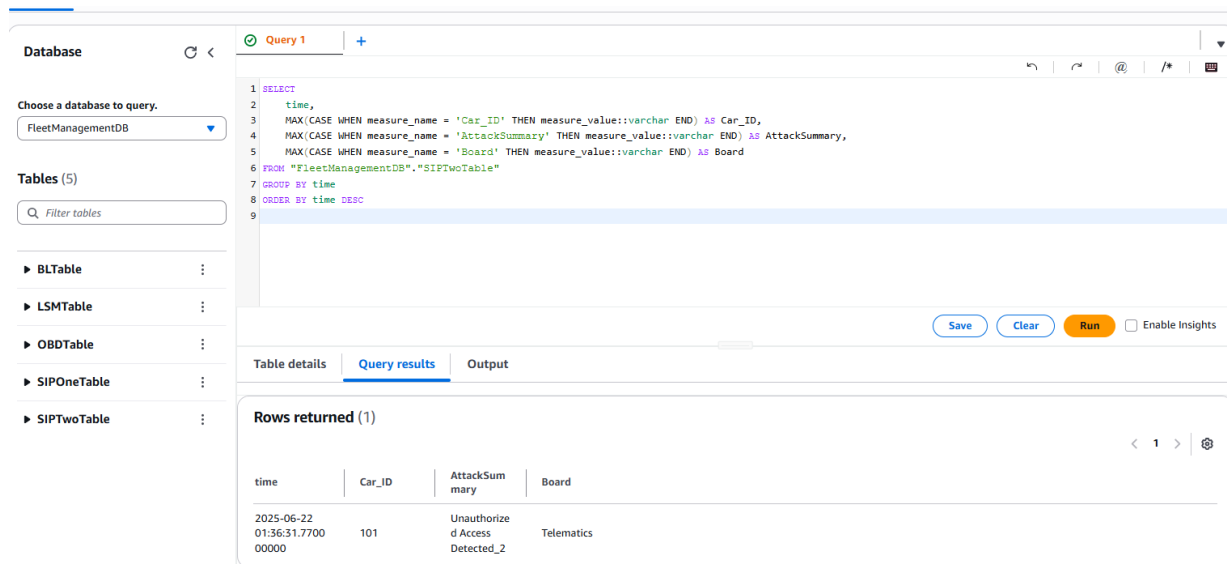


Figure 23 Query Results

Then we have created a script that creates a dashboard that displays live SomeIP 1 & SomeIP 2 data retrieved from AWS Timestream database.

```
import streamlit as st
import boto3
import pandas as pd
from botocore.exceptions import BotoCoreError, ClientError

st.title("🚗 Live SomeIP Dashboard")

# Initialize Timestream client
try:
    client = boto3.client('timestream-query', region_name='us-east-2')
except (BotoCoreError, ClientError) as e:
```

```

st.error(f'Failed to create Timestream client: {e}')
st.stop()

# Define queries
queries = {
    "SOMEIP 1": """
        SELECT
            time,
            MAX(CASE WHEN measure_name = 'Car_ID' THEN measure_value::varchar END) AS
Car_ID,
            MAX(CASE WHEN measure_name = 'AttackSummary' THEN measure_value::varchar END)
AS AttackSummary,
            MAX(CASE WHEN measure_name = 'Board' THEN measure_value::varchar END) AS Board
        FROM "FleetManagementDB"."SIPOneTable"
        GROUP BY time
        ORDER BY time DESC
    """,
    "SOMEIP 2": """
        SELECT
            time,
            MAX(CASE WHEN measure_name = 'Car_ID' THEN measure_value::varchar END) AS
Car_ID,
            MAX(CASE WHEN measure_name = 'AttackSummary' THEN measure_value::varchar END)
AS AttackSummary,
            MAX(CASE WHEN measure_name = 'Board' THEN measure_value::varchar END) AS Board
        FROM "FleetManagementDB"."SIPTwoTable"
        GROUP BY time
        ORDER BY time DESC
    """,
}

# Function to run a Timestream query and return DataFrame
def run_query(query: str):
    try:
        response = client.query(QueryString=query)
        rows = response.get('Rows', [])
        columns = [col['Name'] for col in response.get('ColumnInfo', [])]

        if not rows or not columns:
            return pd.DataFrame()

        data = [[col.get('ScalarValue', '') for col in row['Data']] for row in rows]
        return pd.DataFrame(data, columns=columns)

    except (BotoCoreError, ClientError) as e:
        st.error(f'Query failed: {e}')
        return pd.DataFrame()

```

```
# Query and show data for both SOMEIP tables
for label, sql in queries.items():
    df = run_query(sql)
    st.subheader(f"🔍 Latest {label} Logs")
    if df.empty:
        st.warning(f"No data returned for {label}.")
    else:
        st.dataframe(df)
```

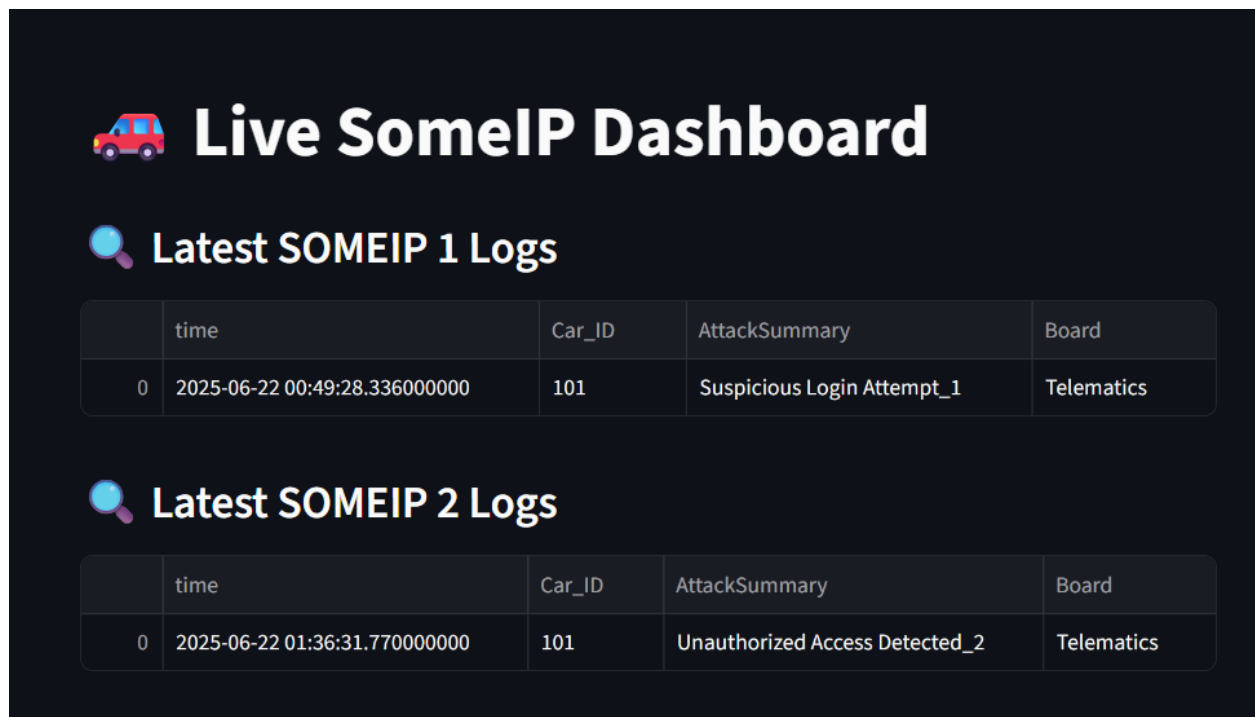


Figure 24 Final Results

Sending Network Real-Time data to the Web App

1. Choose FleetManagementDB

2. Create a Timestream Table

In FleetManagementDB, click "Create table"

Name it: NetworkTable

Leave the default options as it is

Click Create table

3. Create a Thing (Network app)

Thing name: Network_thing

Attach Iot Policy

4. Create IoT Rule to Store data in Timestream

Name the Role: NetworkThingtoTimeStream

Enter this SQL statement to get all the messages from the Network topic.

```
SELECT * FROM 'network_thing'
```

The screenshot shows the configuration for 'Action 1' in the AWS IoT console. The action is 'Timestream table' with the description 'Write a message into a Timestream table'. The 'Database name' is 'FleetManagementDB' and the 'Table name' is 'NetworkTable'. There are three dimensions: 'Car_ID' with value '\${Car_ID}', 'AttackSummary' with value '\${AttackSummary}', and 'Board' with value '\${Board}'. The 'Timestamp value' is '\${time}' and the 'Timestamp unit' is 'MILLISECONDS'. The 'IAM role' is 'demo_rule'. There are buttons for 'Remove', 'Create Timestream database', 'Create Timestream table', 'Add new dimension', 'View', and 'Create new role'.

Dimensions name	Dimension value	Action
Car_ID	\${Car_ID}	Remove
AttackSummary	\${AttackSummary}	Remove
Board	\${Board}	Remove

Figure 25 Role parameters

Test the Fleet Management App

- 1- Run the C++ app that publish messages to AWS IoTCore Service
- 2- Apply MQTT test and make sure that messages are published correctly

Topic filter | [info](#)
The topic filter describes the topic(s) to which you want to subscribe. The topic filter can include MQTT wildcard characters.

network_topic

► **Additional configuration**

Subscribe

Subscriptions

SIPTwo_topic	♡ ×
network_topic	♡ ×

network_topic Pause Clear Export Edit

Message payload

{
 "message": "Hello from AWS IoT console"
}

► **Additional configuration**

Publish

▼ network_topic June 22, 2025, 04:54:48 (UTC+0300)

{
 "AttackSummary": "Minor Security Alert_N",
 "Board": "Telematics",
 "Car_ID": "101",
 "time": "1750557288410"
}

► **Properties**

Figure 26 MQTT test client

Apply this Query test to make sure that Created role (transfer messages from IOTCore to Amazon Timestream DB) works properly:

```
SELECT
  time,
  MAX(CASE WHEN measure_name = 'Car_ID' THEN measure_value::varchar END) AS Car_ID,
  MAX(CASE WHEN measure_name = 'AttackSummary' THEN measure_value::varchar END) AS
AttackSummary,
  MAX(CASE WHEN measure_name = 'Board' THEN measure_value::varchar END) AS Board
FROM "FleetManagementDB"."NetworkTable"
GROUP BY time
ORDER BY time DESC
```

The screenshot shows a SQL query interface. On the left, there's a sidebar with 'Database' and 'Tables (6)' sections. The 'Database' section shows 'FleetManagementDB' selected. The 'Tables' section lists 'BLTable', 'LSMTable', 'NetworkTable', 'OBDDTable', 'SIPOneTable', and 'SIPTwoTable'. The main area shows a SQL query for 'Query 1' with the following text:

```

1 SELECT
2   time,
3   MAX(CASE WHEN measure_name = 'Car_ID' THEN measure_value::varchar END) AS Car_ID,
4   MAX(CASE WHEN measure_name = 'AttackSummary' THEN measure_value::varchar END) AS AttackSummary,
5   MAX(CASE WHEN measure_name = 'Board' THEN measure_value::varchar END) AS Board
6 FROM "FleetManagementDB"."NetworkTable"
7 GROUP BY time
8 ORDER BY time DESC
9

```

Below the query, there are buttons for 'Save', 'Clear', 'Run', and 'Enable Insights'. The 'Query results' tab is active, showing 'Rows returned (2)'. The results are displayed in a table with the following data:

time	Car_ID	AttackSummary	Board
2025-06-22 01:54:48.410000000	101	Minor Security Alert_N	Telematics
2025-06-22 01:54:43.410000000	101	Minor Security Alert_N	Telematics

Figure 27 Query Results

Then we have created a script that creates a dashboard that displays live Network data retrieved from AWS Timestream database.

```

import streamlit as st
import boto3
import pandas as pd
from botocore.exceptions import BotoCoreError, ClientError

st.title("🚗 Live Network Dashboard")

# Initialize Timestream client
try:
    client = boto3.client('timestream-query', region_name='us-east-2')
except (BotoCoreError, ClientError) as e:
    st.error(f'Failed to create Timestream client: {e}')
    st.stop()

# SQL Query
QUERY = """
SELECT
    time,
    MAX(CASE WHEN measure_name = 'Car_ID' THEN measure_value::varchar END) AS Car_ID,
    MAX(CASE WHEN measure_name = 'AttackSummary' THEN measure_value::varchar END) AS
AttackSummary,
    MAX(CASE WHEN measure_name = 'Board' THEN measure_value::varchar END) AS Board
FROM "FleetManagementDB"."NetworkTable"
GROUP BY time
ORDER BY time DESC

```

```

"""

# Execute query
try:
    response = client.query(QueryString=QUERY)
    rows = response.get('Rows', [])
    columns = [col['Name'] for col in response.get('ColumnInfo', [])]

    if not rows or not columns:
        st.warning("No data returned from Timestream.")
        st.stop()

    # Parse rows
    data = [[col.get('ScalarValue', '') for col in row['Data']] for row in rows]
    df = pd.DataFrame(data, columns=columns)

except (BotoCoreError, ClientError) as e:
    st.error(f"Query failed: {e}")
    st.stop()

# Display table
st.subheader("🔍 Latest Network Logs")
st.dataframe(df)

```

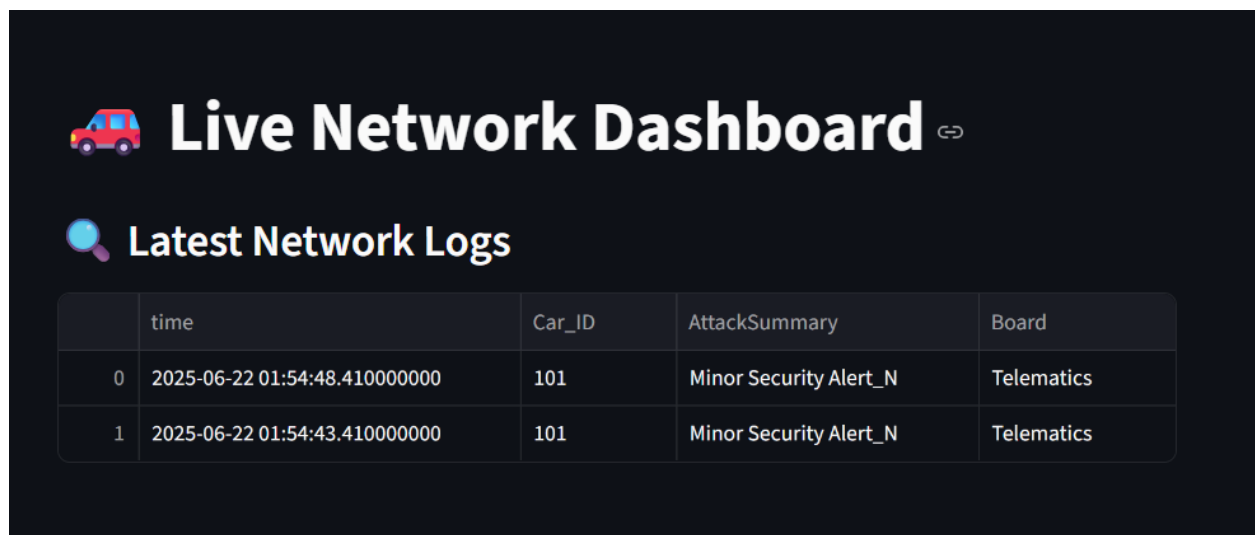


Figure 28 Final Results

References

[1] <https://youtu.be/o5Wt0QS7Oco?si=qk6JGoD6jSdx1UYm>

[2] <https://docs.streamlit.io/>

[3] https://drive.google.com/drive/u/1/folders/1GE_jTglje7QeSanW0_m_X94nfTSYZ-1I?fbclid=IwAR0F2UZ112H90Yk42J0l3rFx-w11fEfEv6AkbURNaXNM81Kf1-2He8VKFao